



GNU grep cheatsheet



ads via Carbon

Fast implementation, a customizable login experience, and flexible user journeys. Try Auth0 for free

Usage

```
grep <options> pattern <file...>
```

Pattern options

```
-F, --fixed-strings  # list of fixed strings
-G, --basic-regexp   # basic regular expression (default)
-E, --extended-regexp # extended regular expression
-P, --perl-regexp    # perl compatible regular expression
```

Output Options

```
-c, --count          # print the count of matching lines. suppresses normal output
--color[=WHEN]      # applies color to the matches. WHEN is never, always, or auto
-m, --max-count=NUM  # stop reading after max count is reached
-o, --only-matching  # only print the matched part of a line
-q, --quiet, --silent # suppress error messages about nonexistent or unreadable files
-s, --no-messages    # suppress error messages about nonexistent or unreadable files
```

Context Options

```
-B NUM, --before-context=NUM # print NUM lines before a match
-A NUM, --after-context=NUM  # print NUM lines after a match
-C NUM, -NUM, --context=NUM  # print NUM lines before and after a match
```

Examples

```
# Case insensitive: match any line in foo.txt
# that contains "bar"
grep -i bar foo.txt

# match any line in bar.txt that contains
# either "foo" or "oof"
grep -E "foo|oof" bar.txt

# match anything that resembles a URL in
# foo.txt and only print out the match
grep -oE "https?://([\\w+\\-]?)+\\.?" foo.txt

# can also be used with pipes:
# match any line that contains "export" in
# .bash_profile, pipe to another grep that
# matches any of the first set of matches
# containing "PATH"
grep "export" .bash_profile | grep "PATH"

# follow the tail of server.log, pipe to grep
# and print out any line that contains "error"
# and include 5 lines of context
tail -f server.log | grep -iC 5 error
```

Matching options

```
-e, --regexp=PATTERN
-f, --file=FILE
-i, --ignore-case
-v, --invert-match
-w, --word-regexp
-x, --line-regexp
```

Expressions

Basic Regular Expressions (BRE)

In BRE, these characters have a special meaning unless they are escaped with a backslash:

```
^ $ . * [ ] \
```

However, these characters do not have any special meaning unless they are escaped with a backslash:

```
? + { } | ( )
```

Extended Regular Expressions (ERE)

ERE gives all of these characters a special meaning unless they are escaped with a backslash:

```
^ $ . * + ? [ ] ( ) | { }
```

Perl Compatible Regular Expressions (PCRE)

PCRE has even more options such as additional anchors and character classes, lookahead/lookbehind, conditional expressions, comments, and more. See the [regex cheatsheet](#).

► 0 Comments for this cheatsheet. [Write yours!](#)





Over 358 curated
cheatsheets, by developers
for developers.

[Devhints home](#)

Cron
cheatsheet

httpie
cheatsheet

composer
cheatsheet

Homebrew
cheatsheet

**adb (Android Debug
Bridge)**
cheatsheet

Fish shell
cheatsheet

Elixir
cheatsheet

React.js
cheatsheet

Vim
cheatsheet

ES2015+
cheatsheet

Vimdiff
cheatsheet

Vim scripting
cheatsheet